

Mise à jour OTA pour ESP32

Fonction `void setupOTA()` prête à l'emploi, lignes à intégrer, procédure et lexique détaillé

1. Principe

L'OTA (*Over-The-Air*) permet de téléverser un nouveau programme dans l'ESP32 **par le WiFi**, sans câble USB. C'est idéal pour une carte déjà montée dans une maquette ou un robot. L'ESP32 reçoit le nouveau firmware dans une zone mémoire libre, vérifie qu'il est intact, puis redémarre dessus. L'ancien programme n'est écrasé qu'une fois le nouveau validé.

Règle d'or : le programme qui contient l'OTA doit être téléversé **une première fois par USB**. Chaque nouveau programme envoyé par OTA doit lui aussi contenir le code OTA, sinon la carte ne pourra plus être mise à jour sans fil.

2. Lignes à placer dans votre programme

Quatre emplacements sont concernés. Le tableau résume ; le code complet suit.

Emplacement	Ligne(s) à ajouter	Rôle
En-tête (tout en haut)	<code>#include <WiFi.h></code> <code>#include <ESPmDNS.h></code> <code>#include <WiFiUdp.h></code> <code>#include <ArduinoOTA.h></code>	1 Charge les bibliothèques WiFi, nom réseau et OTA.
Avant <code>setup()</code>	Les constantes <code>WIFI_SSID</code> , <code>WIFI_PASSWORD</code> , <code>OTA_HOSTNAME</code> , <code>OTA_PASSWORD</code> + les fonctions <code>connexionWiFi()</code> et <code>setupOTA()</code>	2 Paramètres et fonctions réutilisables.
Dans <code>setup()</code>	<code>connexionWiFi();</code> <code>setupOTA();</code>	3 Connexion au routeur puis démarrage du service OTA (dans cet ordre).
Dans <code>loop()</code>	<code>ArduinoOTA.handle();</code>	4 Écoute en permanence les demandes de mise à jour. Indispensable.

3. Code complet commenté

```
/*
 * =====
 * ESP32 - Mise à jour OTA (Over-The-Air) via ArduinoOTA
 * =====
 * 1er téléversement : par câble USB (obligatoire).
 * Ensuite : Arduino IDE > Outils > Port > port réseau "esp32-xxx".
 * Schéma de partition : doit contenir 2 zones OTA
 * (ex. "Default 4MB with spiffs" ou "Minimal SPIFFS").
 * =====
 */

#include <WiFi.h>           // Gestion du WiFi de l'ESP32
#include <ESPmDNS.h>         // Nom réseau "esp32-xxx.local"
#include <WiFiUdp.h>         // Transport UDP utilisé par ArduinoOTA
#include <ArduinoOTA.h>      // Bibliothèque OTA

// ----- PARAMÈTRES À ADAPTER -----
const char* WIFI_SSID      = "MonReseauWiFi";
const char* WIFI_PASSWORD  = "MonMotDePasseWiFi";
const char* OTA_HOSTNAME   = "esp32-lunokhod"; // nom visible dans l'IDE
const char* OTA_PASSWORD   = "ota1234";       // mot de passe OTA

// ----- Connexion WiFi (mode station) -----
void connexionWiFi() {
    WiFi.mode(WIFI_STA);           // ESP32 = client du routeur
    WiFi.setHostname(OTA_HOSTNAME);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    Serial.print("Connexion WiFi");
    unsigned long debut = millis();
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
        if (millis() - debut > 20000) { // 20 s sans succès -> redémarrage
            Serial.println("\nEchec WiFi, redemarrage...");
            ESP.restart();
        }
    }
    Serial.print("\nConnecte ! IP : ");
    Serial.println(WiFi.localIP());
}

// ----- Configuration OTA -----
void setupOTA() {
    ArduinoOTA.setHostname(OTA_HOSTNAME); // nom mDNS
    ArduinoOTA.setPassword(OTA_PASSWORD); // protège les mises à jour
    ArduinoOTA.setPort(3232);             // port par défaut ESP32

    // Début de la mise à jour
    ArduinoOTA.onStart([]() {
        String type = (ArduinoOTA.getCommand() == U_FLASH) ? "programme" : "systeme de fichiers";
        // Si LittleFS/SPIFFS est mis à jour : LittleFS.end(); ici
        Serial.println("OTA : debut mise a jour du " + type);
    });

    // Fin de la mise à jour
    ArduinoOTA.onEnd([]() {
        Serial.println("\nOTA : termine, redemarrage");
    });

    // Progression (en %)
    ArduinoOTA.onProgress([](unsigned int progression, unsigned int total) {
        Serial.printf("OTA : %u%%\r", (progression * 100) / total);
    });

    // Gestion des erreurs
    ArduinoOTA.onError([](ota_error_t erreur) {
        Serial.printf("OTA erreur [%u] : ", erreur);
        if (erreur == OTA_AUTH_ERROR) Serial.println("Mot de passe refuse");
    });
}
```

```

    else if (erreur == OTA_BEGIN_ERROR)    Serial.println("Echec au démarrage");
    else if (erreur == OTA_CONNECT_ERROR) Serial.println("Echec de connexion");
    else if (erreur == OTA_RECEIVE_ERROR) Serial.println("Echec de reception");
    else if (erreur == OTA_END_ERROR)      Serial.println("Echec de finalisation");
  });

  ArduinoOTA.begin();                      // démarre le service OTA
  Serial.println("OTA pret.");
}

// ----- Programme principal -----
void setup() {
  Serial.begin(115200);
  connexionWiFi(); 3
  setupOTA();
  // ... votre initialisation (capteurs, moteurs, etc.)
}

void loop() {
  ArduinoOTA.handle(); // OBLIGATOIRE : écoute les demandes OTA
  // ... votre code, SANS delay() long (utiliser millis())
}

```

4. Procédure pas à pas (Arduino IDE)

1. Installer le support ESP32 : *Fichier > Préférences*, URL de cartes supplémentaires, puis *Gestionnaire de cartes > « esp32 by Espressif »*. ArduinoOTA est inclus, rien d'autre à installer.
2. Renseigner SSID, mot de passe WiFi, nom d'hôte et mot de passe OTA dans le code.
3. Choisir un **schéma de partition avec OTA** : *Outils > Partition Scheme > « Default 4MB with spiffs »* (ou « Minimal SPIFFS » si le programme est gros). À éviter : « Huge APP (No OTA) ».
4. Téléverser **par USB** et ouvrir le moniteur série (115200 bauds) : l'adresse IP et « OTA pret. » doivent s'afficher.
5. Débrancher l'USB (alimentation externe possible). Dans *Outils > Port*, un **port réseau** « esp32-lunokhod at 192.168.x.x » apparaît.
6. Sélectionner ce port et téléverser normalement. L'IDE demande le mot de passe OTA.
7. À la fin, l'ESP32 redémarre seul sur le nouveau programme.

Variante PlatformIO (fichier platformio.ini)

```

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
upload_protocol = espota
upload_port = 192.168.1.50 ; IP de l'ESP32 ou esp32-lunokhod.local
upload_flags =
  --port=3232
  --auth=ota1234 ; = OTA_PASSWORD

```

Variante ligne de commande (espota.py)

```
python espota.py -i 192.168.1.50 -p 3232 -a ota1234 -f programme.ino.bin
```

Le fichier .bin s'obtient via *Croquis > Exporter les binaires compilés*. espota.py se trouve dans le dossier *tools* du paquet ESP32.

5. Précautions et dépannage

Symptôme	Cause probable	Solution
Aucun port réseau dans l'IDE	PC et ESP32 sur des réseaux différents, mDNS bloqué, IDE lancé avant l'ESP32	Même réseau WiFi ; redémarrer l'IDE ; autoriser l'IDE/Python dans le pare-feu ; utiliser l'IP directement.
« No response from device »	Port 3232 bloqué, <code>handle()</code> rarement appelé	Ouvrir le pare-feu ; supprimer les <code>delay()</code> longs dans <code>loop()</code> .
<code>OTA_AUTH_ERROR</code>	Mot de passe différent	Utiliser exactement <code>OTA_PASSWORD</code> .
<code>OTA_BEGIN_ERROR</code>	Programme trop gros ou partition sans OTA	Changer le schéma de partition.
<code>OTA_RECEIVE_ERROR</code>	WiFi faible, coupure en cours de transfert	Rapprocher du routeur, relancer.
Carte injoignable après une mise à jour	Le nouveau programme ne contient pas l'OTA ou plante au démarrage	Retéléverser par USB un programme avec OTA.

Sécurité (robots, moteurs) : dans `onStart()`, arrêtez les moteurs et mettez les sorties dans un état sûr : pendant le transfert, le reste du programme tourne au ralenti.

6. Lexique détaillé

OTA (Over-The-Air)

« Par les airs ». Mise à jour du programme via une liaison sans fil (ici le WiFi) au lieu du câble USB/série.

Firmware

Le programme compilé exécuté par le microcontrôleur. C'est ce que l'on remplace lors d'une mise à jour OTA (fichier *.bin*).

Téléversement (upload)

Envoi du firmware compilé depuis l'ordinateur vers la mémoire flash de l'ESP32.

Mémoire flash

Mémoire non volatile (conservée hors tension) où sont stockés le firmware et les fichiers. Typiquement 4 Mo sur un ESP32.

Partition / schéma de partition

Découpage de la flash en zones : **app0** et **app1** (deux emplacements programme), **otadata**, **nvs**, **spiffs**... L'OTA exige deux zones programme : l'une tourne pendant que l'autre reçoit la nouvelle version.

otadata

Petite partition qui indique au bootloader quelle zone (app0 ou app1) démarrer.

Bootloader

Petit programme lancé au démarrage de l'ESP32 ; il lit otadata et démarre le bon firmware.

Rollback

Retour automatique à l'ancien firmware si le nouveau est invalide (contrôle d'intégrité échoué).

ArduinoOTA

Bibliothèque incluse dans le core ESP32 qui gère l'OTA depuis l'IDE Arduino ou PlatformIO : écoute réseau, authentification, écriture en flash, redémarrage.

ArduinoOTA.begin()

Démarré le service OTA. À appeler **après** la connexion WiFi et **après** les réglages (setHostname, setPassword...).

ArduinoOTA.handle()

Vérifie s'il y a une demande de mise à jour et la traite. À appeler en boucle dans loop(), le plus souvent possible.

setHostname()

Définit le nom réseau de la carte (ex. esp32-lunokhod). Il apparaît dans la liste des ports de l'IDE et répond à *esp32-lunokhod.local*.

setPassword() / setPasswordHash()

Protège l'OTA par mot de passe. setPasswordHash() accepte l'empreinte MD5 du mot de passe, pour ne pas l'écrire en clair dans le code.

setPort()

Port réseau d'écoute OTA. Par défaut **3232** sur ESP32.

Callback

Fonction qu'on fournit à une bibliothèque et qu'elle appelle d'elle-même au bon moment. Ici : onStart, onEnd, onProgress, onError.

Fonction lambda [] () { }

Fonction anonyme écrite directement en argument, sans nom. Syntaxe C++ utilisée pour les callbacks OTA.

onStart()

Appelé au début du transfert : idéal pour arrêter moteurs, fermer les fichiers (LittleFS.end()).

onProgress()

Appelé régulièrement avec les octets reçus et le total : permet d'afficher un pourcentage ou d'animer une LED.

onEnd()

Appelé quand le transfert est terminé avec succès, juste avant le redémarrage.

onError() / ota_error_t

Appelé en cas d'échec avec un code : OTA_AUTH_ERROR (mot de passe), OTA_BEGIN_ERROR (démarrage/place), OTA_CONNECT_ERROR (connexion), OTA_RECEIVE_ERROR (réception), OTA_END_ERROR (finalisation).

getCommand() / U_FLASH / U_SPIFFS

Indique ce qui est mis à jour : U_FLASH = le programme, U_SPIFFS = le système de fichiers.

SPIFFS / LittleFS

Systèmes de fichiers stockés en flash (pages web, configurations...). Ils peuvent aussi être mis à jour par OTA.

WiFi.mode(WIFI_STA)

Mode « station » : l'ESP32 se connecte à un routeur existant comme un téléphone. (WIFI_AP : l'ESP32 crée son propre réseau.)

SSID

Nom du réseau WiFi auquel se connecter.

WL_CONNECTED

Valeur renvoyée par WiFi.status() quand la connexion est établie.

Adresse IP

Identifiant de la carte sur le réseau local (ex. 192.168.1.50), affiché par WiFi.localIP().

mDNS (ESPmDNS)

Service qui annonce le nom de la carte sur le réseau local (*nom.local*) : l'IDE découvre ainsi l'ESP32 sans connaître son IP.

UDP / TCP

Protocoles réseau. L'IDE envoie une invitation en UDP ; le firmware est ensuite transféré en TCP (fiable, avec accusés de réception).

espota.py

Script Python fourni avec le core ESP32 qui envoie un .bin par OTA. Utilisé en coulisses par l'IDE et PlatformIO.

MD5

Algorithme d'empreinte utilisé pour vérifier l'intégrité du firmware reçu et pour l'authentification OTA.

millis()

Nombre de millisecondes depuis le démarrage. Permet des temporisations **non bloquantes** (contrairement à delay()) pour que handle() soit appelé souvent.

delay()

Pause bloquante : pendant ce temps, rien d'autre ne s'exécute, OTA compris. À limiter dans loop().

ESP.restart()

Redémarre l'ESP32 par logiciel (utilisé ici si le WiFi ne se connecte pas en 20 s).

Serial.begin(115200)

Ouvre le port série à 115200 bauds pour afficher les messages dans le moniteur série.

#include

Directive qui insère une bibliothèque dans le programme avant la compilation.

void

Type d'une fonction qui ne renvoie aucune valeur : elle exécute des actions (ex. void setupOTA()).

const char*

Chaîne de caractères constante (texte non modifiable), utilisée pour les identifiants et mots de passe.